
Module 09 — Distributed Data with Hazelcast

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 What Is Hazelcast?	2
2.2 Embedded or Client-Server?	2
3. Step-by-Step Examples	3
3.1 An Embedded Member	3
3.2 The IMap	4
3.3 Queries	5
3.4 Spring Cache on Hazelcast	5
3.5 Concurrent Updates: Key Locks and Entry Processors	6
3.6 Client-Server with a Near Cache	8
3.7 A Cluster of Two Members	9
3.8 Testing	10
4. Common Mistakes and Best Practices	10
5. Summary	11
6. Further Reading	11

1. Learning Goals

By the end of this module you will be able to:

- Explain the difference between an embedded Hazelcast member and a client-server setup
- Start Hazelcast from Spring Boot in both modes
- Store, expire and query data with a distributed IMap
- Use Hazelcast as the store behind Spring's @Cacheable
- Update shared data safely under concurrency with key locks and entry processors
- Speed up reads with a near cache and know its trade-offs
- Run a two-member cluster and test Hazelcast code with Testcontainers

Prerequisites: Module 08 (caching with Spring) · **Estimated time:** 4 hours · **Docker required** only for the client-server part

2. Concepts

2.1 What Is Hazelcast?

Hazelcast is an in-memory data grid: several JVMs (**members**) join into a **cluster** and share data structures such as maps, queues and topics. The data of a map is split into **partitions** (271 by default), and every partition is owned by one member and has backup copies on others. When a member leaves, its backups take over.

	Redis (module 08)	Hazelcast
Runs	As a separate server	Inside your JVM (embedded) or as a separate cluster
Data	Strings, lists, sets, sorted sets, hashes	Java-friendly maps, queues, topics... of objects
Scaling	Single node, or Redis Cluster	Members join and split the data automatically
Compute	Lua scripts	Entry processors and queries run where the data is

2.2 Embedded or Client-Server?

Embedded member	Client-server
Hazelcast runs inside every application instance	Applications are clients of a separate Hazelcast cluster
No extra servers. Data lives next to the code, so reads are fastest	Applications and data scale and restart independently
Each application restart moves partitions around	Application restarts do not touch the data
All members need the same classes	Servers need classes only for code that runs on them (entry processors)

In Spring Boot the decision is made by one bean: a `com.hazelcast.config.Config` bean starts an **embedded member**, and connection details for a cluster (from Docker Compose, Testcontainers or a `hazelcast-client.yaml`) create a **client**.

! Important

Since Hazelcast 5.5, the **CP Subsystem** (`FencedLock`, `IAtomicLong`, `ISemaphore`...) is an **Enterprise** feature. In the Community edition these calls throw `UnsupportedOperationException`: CP subsystem is a licensed feature. This module therefore uses `IMap` key locks and entry processors, which are part of the Community edition.

3. Step-by-Step Examples

Start the application. Hazelcast runs **embedded**, so no Docker is needed:

```
./mvnw -pl modules/09-hazelcast/lesson spring-boot:run
```

The tour (`LessonTour`) runs every example below once and prints the results. An embedded member keeps the JVM running like a web server does: stop it with `Ctrl+C`.

3.1 An Embedded Member

A `Config` bean is all Boot needs. `HazelcastAutoConfiguration` builds the member from it and registers the `HazelcastInstance` bean:

```
@Configuration(proxyBeanMethods = false)
@Profile("!client")
public class MemberConfiguration {
```

```

@Bean
Config hazelcastConfig() {
    Config config = new Config();
    config.setClusterName("bookstore");
    config.setProperty("hazelcast.phone.home.enabled", "false");
    var join = config.getNetworkConfig().getJoin();
    join.getAutoDetectionConfig().setEnabled(false);           // a single member: do
↪ not look for others
    join.getMulticastConfig().setEnabled(false);
    config.addMapConfig(new MapConfig("prices").setTimeToLiveSeconds(600)); //
↪ cached prices expire
    return config;
}
}

```

- `setClusterName`: only members with the same name form a cluster.
- Join settings decide how members find each other: multicast, TCP/IP member lists, or cloud discovery (Kubernetes, AWS...). A single member does not need any.
- `@Profile("!client")`: in the `client` profile (section 3.6) the application must not start a member.

★ Tip

Instead of a `Config` bean, you can put a `hazelcast.yaml` on the classpath or point `spring.hazelcast.config` at a file. Boot picks it up the same way.

3.2 The IMap

An `IMap` behaves like a `ConcurrentMap`, but its entries are spread over the partitions of the cluster:

```

@Service
public class BookCatalog {

    private final IMap<String, Book> books;

    public BookCatalog(HazelcastInstance hazelcast) {
        this.books = hazelcast.getMap("books");           // created on first
↪ use
    }

    public void save(Book book) {
        books.set(book.isbn(), book);                     // set(): like put()
↪ without returning the old value
    }
}

```

```

public boolean addIfAbsent(Book book) {
    return books.putIfAbsent(book.isbn(), book) == null;    // atomic across
    ↪ the whole cluster
}

public void saveFor(Book book, Duration timeToLive) {
    books.set(book.isbn(), book, timeToLive.toMillis(), TimeUnit.MILLISECONDS);    //
    ↪ this entry expires on its own
}

public Optional<Book> find(String isbn) {
    return Optional.ofNullable(books.get(isbn));
}

```

- set stores a value without returning the old one. It saves a network round trip compared to put.
- putIfAbsent is atomic across the whole cluster.
- The four-argument set gives one entry its own time to live. MapConfig.setTimeToLiveSeconds does the same for a whole map.
- Book is a plain record. Hazelcast serializes records with **Compact serialization** without any configuration. The format carries its own schema, so a server member can store and query books without having the Book class.

3.3 Queries

Queries run on every member in parallel, on the data each member owns:

```

public Collection<Book> cheaperThan(BigDecimal limit) {
    return books.values(Predicates.lessThan("price", limit));    // runs on every member,
    ↪ in parallel
}

```

Predicates offers equal, between, like, in, and, or and more. With map.addIndex(IndexType.SORTED, "price"), range queries no longer scan every entry.

The tour output:

```

== 3.3 Query
cheaper than 90: [Effective Java, Java Puzzlers]
Spring in Action after its 2 s TTL: expired

```

3.4 Spring Cache on Hazelcast

The cache annotations are the same as in module 08. Only the store changes. With a HazelcastInstance bean and hazelcast-spring on the classpath, Boot creates a Hazel-

castCacheManager, and every cache is an IMap of the same name:

```
@Service
public class PriceService {

    private final AtomicInteger calls = new AtomicInteger();

    @Cacheable("prices")
    public BigDecimal priceOf(String isbn) {
        calls.incrementAndGet();
        sleep(300); // pretend to ask a
        ↪ slow pricing system
        return new BigDecimal("89.90");
    }
}
```

The time to live of the prices cache comes from its MapConfig in MemberConfiguration (section 3.1).

```
== 3.4 Spring Cache
1st price: 89.90 in 311 ms
2nd price: 89.90 in 0 ms
```

3.5 Concurrent Updates: Key Locks and Entry Processors

Reserving stock means “read the quantity, check it, write it back”. If two callers interleave, both see 1 book left, and the book is sold twice. There are two safe ways.

A key lock. IMap.lock(key) locks one key in the whole cluster. Other keys stay free:

```
public boolean reserveWithLock(String isbn, int quantity) {
    stock.lock(isbn); // cluster-wide lock on
    ↪ this key only
    try {
        int available = available(isbn);
        if (available < quantity) {
            return false;
        }
        stock.set(isbn, available - quantity);
        return true;
    } finally {
        stock.unlock(isbn); // always release, even
        ↪ after an exception
    }
}
```

```

public boolean tryReserve(String isbn, int quantity) throws InterruptedException {
    if (!stock.tryLock(isbn, 100, TimeUnit.MILLISECONDS)) {           // do not wait forever
        ↪ for a busy key
        return false;
    }
    try {
        int available = available(isbn);
        if (available < quantity) {
            return false;
        }
        stock.set(isbn, available - quantity);
        return true;
    } finally {
        stock.unlock(isbn);
    }
}

```

- Always unlock in finally. A lock belongs to the thread that took it.
- tryLock with a timeout gives up instead of waiting forever for a busy key.
- lock(key, leaseTime, unit) releases the lock by itself after the lease time, in case the holder never unlocks.

An entry processor. Instead of bringing the data to the code, send the code to the data. The processor runs on the member that owns the key, and Hazelcast runs only one processor at a time per key. No lock is needed, and it is one network round trip instead of four:

```

public record ReserveStock(int quantity) implements EntryProcessor<String, Integer,
    ↪ Boolean> {

    @Override
    public Boolean process(Map.Entry<String, Integer> entry) {
        Integer available = entry.getValue();
        if (available == null || available < quantity) {
            return false;
        }
        entry.setValue(available - quantity);           // written back by
    ↪ Hazelcast
        return true;
    }
}

```

```

public boolean reserveWithEntryProcessor(String isbn, int quantity) {
    return Boolean.TRUE.equals(stock.executeOnKey(isbn, new ReserveStock(quantity)));
    ↪ // one round trip
}

```

StockReservationTest lets 50 virtual threads buy 10 books at once. Both versions sell exactly 10.

! Warning

An entry processor is executed **on the member**. In client-server mode, its class must be on the servers' classpath, otherwise the call fails. That is why the tour skips the entry processor in the client profile. Prefer entry processors in embedded setups, or deploy their classes to the cluster.

3.6 Client-Server with a Near Cache

Start the application as a **client** of the Hazelcast member from the root `compose.yaml`:

```
./mvnw -pl modules/09-hazelcast/lesson spring-boot:run -Dspring-boot.run.profiles=client
```

The client profile switches Docker Compose support on. Boot starts `hazelcast/hazelcast`, reads its `HZ_CLUSTERNAME`, and provides `HazelcastConnectionDetails`:

```
spring:
  docker:
    compose:
      enabled: true # starts hazelcast/hazelcast and provides
                  ↪ HazelcastConnectionDetails
```

With connection details alone, Boot would build the client itself. Here we build it ourselves to add a **near cache**, a local copy of the books map inside the client:

```
@Configuration(proxyBeanMethods = false)
@Profile("client")
public class ClientConfiguration {

    @Bean(destroyMethod = "shutdown")
    HazelcastInstance hazelcastInstance(HazelcastConnectionDetails connection) {
        ClientConfig config = connection.getClientConfig(); // address + cluster
        ↪ name from Docker Compose / Testcontainers
        config.addNearCacheConfig(new NearCacheConfig("books")
            .setInMemoryFormat(InMemoryFormat.OBJECT) // keep deserialized
            ↪ objects: fastest reads
            .setTimeToLiveSeconds(60) // bound how stale a
            ↪ local copy can get
            .setInvalidateOnChange(true)); // the cluster tells
            ↪ us when an entry changes
        return HazelcastClient.newHazelcastClient(config);
    }
}
```

```
}
```

- The first get goes to the server. Repeated reads of the same key are answered from local memory. In the exercise, a near-cached read took about 1.5 μ s and a read from the member about 70 μ s.
- `setInvalidateOnChange(true)`: when an entry changes in the cluster, the member tells the clients to drop their copy.
- Invalidations are **batched**: by default a member sends them every 10 seconds or every 100 changes. A near cache is therefore eventually consistent. Use it for data that is read much more often than it changes.

3.7 A Cluster of Two Members

`TwoMemberClusterTest` starts two members in one JVM. They find each other over TCP/IP on 127.0.0.1:

```
private static Config memberConfig() {
    Config config = new Config();
    config.setClusterName("cluster-demo"); // only members
    ↪ with this name join
    config.setProperty("hazelcast.phone.home.enabled", "false");
    var join = config.getNetworkConfig().getJoin();
    join.getAutoDetectionConfig().setEnabled(false);
    join.getMulticastConfig().setEnabled(false);
    join.getTcpIpConfig().setEnabled(true).addMember("127.0.0.1"); // find each
    ↪ other on this machine
    config.getMapConfig("books").setBackupCount(1); // one backup
    ↪ copy on another member
    return config;
}
```

The tests show three things:

1. Both members see a cluster of size 2.
2. An entry written on one member can be read on the other. The caller does not know which member owns the key.
3. After one member shuts down, all 100 entries are still there, because the backups on the other member become the owners.

★ Tip

In production, members rarely use a fixed member list. On Kubernetes, for example, the Hazelcast Kubernetes discovery plugin finds the members through the Kubernetes API.

3.8 Testing

- Embedded code is tested with `@SpringBootTest`: every test context starts its own member. Keep the join disabled so that test members never find each other.
- Client-server code is tested against a real member in Docker. Testcontainers has no dedicated Hazelcast container class, so a `GenericContainer` is used, and `@ServiceConnection` needs the image name:

```
@TestConfiguration(proxyBeanMethods = false)
static class HazelcastServer {

    static final GenericContainer<?> MEMBER = new
        ↪ GenericContainer<>("hazelcast/hazelcast:5.5.0") // as compose.yaml
            .withEnv("HZ_CLUSTERNAME", "bookstore") // Boot reads it to set
            ↪ the client's cluster name
            .withExposedPorts(5701);

    @Bean
    @ServiceConnection(name = "hazelcast/hazelcast") // generic container: tell
        ↪ Boot what it runs
    GenericContainer<?> hazelcastMember() {
        return MEMBER;
    }
}
```

! Warning

Without `name = "hazelcast/hazelcast"`, the test fails with `ConnectionDetailsNotFoundException`, because Boot cannot tell what a generic container runs.

4. Common Mistakes and Best Practices

!! Caution

Never use “read, check, write” on shared data without a lock or an entry processor. Every instance of your application may run the same code at the same moment.

- **Do:** disable multicast and list your members explicitly (or use a discovery plugin). Multicast lets unrelated applications on the same network join your cluster.
- **Don't:** forget `unlock` in `finally`. A key that stays locked blocks every later caller.
- **Do:** lock several keys in one fixed order (e.g. sorted). Two callers that lock the same keys in opposite order wait for each other forever: a deadlock (exercise 2).
- **Don't:** store huge objects in one entry. A partition is moved as a whole when members join or leave.

- **Do:** give caches a time to live, and give near caches a short one if slightly stale data would hurt.
- **Don't:** expect FencedLock or IAtomicLong to work in the Community edition since 5.5.

5. Summary

- Hazelcast shares data between JVMs. Partitions have owners and backups, so the cluster survives the loss of a member.
- In Boot, a Config bean gives an embedded member, and HazelcastConnectionDetails give a client.
- IMap supports TTL per entry, atomic operations such as putIfAbsent, and parallel queries with Predicates.
- @Cacheable works unchanged: every cache is an IMap.
- Key locks and entry processors make concurrent updates safe. Entry processors run where the data is.
- A near cache makes repeated client reads local and fast, but only eventually consistent.
- In tests, embedded members need no Docker. Client-server tests use a GenericContainer with @ServiceConnection(name = "hazelcast/hazelcast").

Next step: [Exercises](#)

6. Further Reading

- [Spring Boot — Hazelcast](#)
- [Hazelcast — Distributed Map](#) · [Locking Maps](#)
- [Hazelcast — Entry Processor](#)
- [Hazelcast — Near Cache](#)
- [Hazelcast — Compact Serialization](#)
- [Hazelcast — CP Subsystem](#) · [5.5 Release Notes](#)